

Universidad Técnica Estatal de Quevedo
Facultad de Ciencias de la Computación y Diseño Digital
Carrera de Ingeniería de Software

Asignatura: Aplicaciones Web

Especificación de Requisitos de Software (SRS)

Conforme a ISO/IEC/IEEE 29148:2018

Proyecto: BIOPET — Sistema Integral de Gestión Veterinaria

Versión del documento: v0.9.0-rc (Tercera Entrega)

Reemplaza a: SRS v0.3.0 (Entrega 1A)

Integrantes:

Beltrán Montiel Fred Adrián — C.I. 1250300876
Mariscal Cabrera Jaime Josué — C.I. 1250710835
Taipe Mora Zaida Melissa — C.I. 0604907956

Docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.
Periodo Académico: 2026-2027 PPA — Quevedo, Los Ríos, Ecuador

Control de versiones del documento

Versión	Fecha	Autor(es)	Descripción del cambio
v0.3.0	04-jun-2026	Equipo BIOPET	Versión original del SRS, Entrega 1A. Corpus inicial de requisitos, stack propuesto ASP.NET Core 8.
v0.9.0-rc	24-jul-2026	Equipo BIOPET	Actualización para la Tercera Entrega. Migración documental del stack a Java 21 / Spring Boot 3.2 (cambio real ocurrido en la Entrega 1B). Requisitos reescritos conforme a ISO/IEC/IEEE 29148:2018 e INCOSE v4. Alcance de la iteración actual acotado a los módulos de Autenticación y Mascotas, efectivamente implementados. Requisitos restantes reclasificados como pendientes (Should/Could) con trazabilidad explícita. Ver docs/requisitos/CHANGELOG-REQ.md y docs/requisitos/CAMBIOS-SRS.md para el detalle completo.

Tabla de contenido

Control de versiones del documento	2
Tabla de contenido.....	3
1. Introducción	4
1.1. Propósito	4
1.2. Alcance del producto.....	4
1.3. Definiciones, acrónimos y abreviaturas.....	4
1.4. Referencias.....	5
1.5. Visión general del documento	5
2. Descripción global del producto	6
2.1. Perspectiva del producto.....	6
2.2. Cambio de plataforma tecnológica respecto a la Entrega 1A	6
2.3. Funciones del producto	6
2.3.1. Funciones implementadas y verificadas (v0.9.0-rc).....	6
2.3.2. Funciones planificadas y pendientes de implementación	6
2.4. Características de los usuarios.....	6
2.5. Restricciones	7
2.6. Supuestos y dependencias	7
3. Requisitos específicos.....	8
3.1. Requisitos funcionales — Módulo de Autenticación y Autorización (implementado).....	8
3.2. Requisitos funcionales — Módulo de Mascotas (implementado)	9
3.3. Requisitos funcionales — Módulos pendientes (heredados de la Entrega 1A, no implementados)....	11
3.4. Requisitos no funcionales	13
4. Arquitectura de la aplicación web	17
4.1. Diagrama de contexto (C4 — Nivel 1).....	17
4.2. Diagrama de contenedores (C4 — Nivel 2).....	17
4.3. Diagrama de clases	18
4.4. Diagrama de secuencia: flujo de autenticación JWT	18
4.5. Descripción de capas y tecnologías (actualizada)	18
5. Modelo de base de datos	20
5.1. Diccionario de datos — Entidades implementadas	20
5.1.1. usuarios	20
5.1.2. mascotas.....	21
5.2. Entidades pendientes de materialización	21
6. Matriz de trazabilidad (resumen)	22
7. Observaciones abiertas e información pendiente	23

1. Introducción

1.1. Propósito

El presente documento especifica los requisitos de software del sistema BIOPET, una plataforma web integral para la gestión clínica y administrativa de clínicas veterinarias de pequeña y mediana escala en Ecuador. Esta versión (v0.9.0-rc) actualiza y reemplaza la Especificación de Requisitos de Software (SRS) producida en la Entrega 1A (v0.3.0), consolidando los requisitos con base en el estado real del sistema alcanzado hasta la Entrega 1B y estableciendo la línea base de requisitos para el estado release candidate exigido por la Tercera Entrega del Proyecto Fin de Curso (PFC).

El documento sigue la plantilla de Software Requirements Specification (SRS) de la norma ISO/IEC/IEEE 29148:2018 y las quince características de calidad de requisitos individuales y de conjuntos definidas por el INCOSE Guide to Writing Requirements v4.

1.2. Alcance del producto

BIOPET centraliza la información clínica, administrativa y de facturación de una clínica veterinaria, sustituyendo el uso de hojas de cálculo, mensajería informal y procesos manuales. El sistema contempla cuatro roles: Administrador, Veterinario, Auxiliar y Dueño de mascota.

Alcance funcional consolidado a la fecha de esta versión (v0.9.0-rc): módulo de autenticación y autorización stateless basado en JWT (registro, inicio de sesión, renovación y cierre de sesión con revocación mediante lista negra en Redis), y CRUD completo de la entidad Mascota con control de acceso por rol. Este es el núcleo verificado con código, pruebas automatizadas y evidencia empírica preliminar.

Los módulos de historial clínico, citas veterinarias, facturación digital, telemetría IoT y recomendaciones asistidas por IA, descritos como parte de la visión del producto en la Entrega 1A, permanecen vigentes como requisitos Should/Could del sistema y se encuentran en estado pendiente de implementación. Esta especificación los mantiene documentados con el mismo rigor que los requisitos ya implementados para preservar la trazabilidad hacia el diseño de base de datos y los wireframes producidos en la Entrega 1A, pero declara explícitamente su estado en la matriz de trazabilidad (docs/trazabilidad/matriz.csv), conforme lo exige el bloque A.3.3 de la Guía de la Tercera Entrega.

1.3. Definiciones, acrónimos y abreviaturas

Término	Definición
SRS	Software Requirements Specification — Especificación de Requisitos de Software.
RF / RNF	Requisito Funcional / Requisito No Funcional.
JWT	JSON Web Token — RFC 7519. Formato de token firmado usado para autenticación stateless.
JTI	JWT ID — identificador único de un token, usado como clave de revocación.
RBAC	Role-Based Access Control — control de acceso basado en roles.
ORM	Object-Relational Mapping. En BIOPET, Spring Data JPA sobre Hibernate.
MoSCoW	Técnica de priorización de requisitos: Must, Should, Could, Won't.
INVEST	Independent, Negotiable, Valuable, Estimable, Small, Testable — criterios de calidad de historias de usuario.
C4	Modelo de diagramación arquitectónica de Simon Brown (Contexto, Contenedores, Componentes, Código).
ADR	Architecture Decision Record — registro de decisión arquitectónica, plantilla de Nygard.
TTL	Time To Live — tiempo de vida de una entrada en caché o de un token.
OWASP	Open Worldwide Application Security Project.

1.4. Referencias

- ISO/IEC/IEEE 29148:2018 — Systems and software engineering — Life cycle processes — Requirements engineering.
- INCOSE Guide to Writing Requirements v4, INCOSE-TP-2010-006-04 (2023).
- RFC 7519 — JSON Web Token (JWT), IETF, 2015.
- RFC 7807 — Problem Details for HTTP APIs, IETF, 2016.
- OWASP Top 10:2021 — The Ten Most Critical Web Application Security Risks.
- ISO/IEC 25010:2011 — Systems and software quality requirements and evaluation (SQuaRE).
- Guía de la Tercera Entrega del PFC — Aplicaciones Web, UTEQ, PPA 2026-2027.
- SRS de la Entrega 1A — BIOPET (04-jun-2026), documento predecesor de esta versión.
- Informe técnico de la Entrega 1B — BIOPET, Módulo de Autenticación JWT y Acceso a Datos (2026).

1.5. Visión general del documento

La sección 2 presenta la descripción global del producto (perspectiva, funciones principales, usuarios y restricciones). La sección 3 contiene los requisitos específicos: funcionales, no funcionales, de interfaz externa y de calidad, cada uno con identificador, rationale, prioridad MoSCoW, criterio de aceptación medible y método de verificación. La sección 4 documenta la arquitectura y los diagramas vigentes. La sección 5 describe el modelo de datos. La sección 6 resume la matriz de trazabilidad. La sección 7 declara las observaciones abiertas y la información pendiente de completar.

2. Descripción global del producto

2.1. Perspectiva del producto

BIOPET es un producto nuevo, sin dependencia de sistemas heredados. Sustituye la gestión manual (papel, hojas de cálculo, mensajería) por una plataforma web con backend REST y frontend de página única (SPA). El sistema se despliega mediante contenedores Docker y no depende de infraestructura física adicional en la clínica; su arquitectura está preparada para incorporar en fases posteriores servicios externos de IoT (dispositivos de localización) y de Inteligencia Artificial (recomendaciones clínicas), sin que estos formen parte del alcance implementado en esta versión.

2.2. Cambio de plataforma tecnológica respecto a la Entrega 1A

La Entrega 1A seleccionó ASP.NET Core 8 como lenguaje de servidor (ADR-001) por afinidad del equipo con C#. Durante la Entrega 1B el equipo migró la implementación a Java 21 con Spring Boot 3.2, Spring Security 6, Spring Data JPA/Hibernate, PostgreSQL 16, Flyway y Redis 7, decisión que se documenta y sustenta en el repositorio de la Entrega 1B. Este SRS actualiza toda referencia tecnológica del documento original para reflejar la pila efectivamente implementada; ADR-001 queda superado y debe registrarse un nuevo ADR de cambio de pila (ver observación OBS pendiente en la sección 7 de este documento, dado que el repositorio provisto no contiene aún dicho ADR de forma explícita).

2.3. Funciones del producto

Resumen de alto nivel de las funciones que el sistema debe proveer a lo largo de su ciclo de vida, agrupadas por estado de implementación en esta versión.

2.3.1. Funciones implementadas y verificadas (v0.9.0-rc)

- Registro, inicio de sesión, renovación (refresh) y cierre de sesión de usuarios mediante JWT stateless.
- Revocación de tokens mediante lista negra de JTI en Redis con expiración automática.
- Control de acceso basado en roles (ROLE_ADMIN, ROLE_VETERINARIO, ROLE_AUXILIAR, ROLE_DUENO).
- Alta, consulta, actualización y baja lógica de mascotas, con paginación y asociación al dueño.
- Respuesta uniforme de errores conforme a RFC 7807 (ProblemDetails).

2.3.2. Funciones planificadas y pendientes de implementación

- Gestión de historial clínico y atenciones médicas.
- Gestión de citas veterinarias mediante calendario interactivo.
- Facturación digital, registro de pagos y generación de comprobantes PDF.
- Recepción de telemetría de dispositivos de localización IoT y visualización en mapas.
- Recomendaciones clínicas asistidas por IA a partir del historial médico.
- Generación de reportes estadísticos exportables (PDF/Excel) y auditoría de operaciones.

2.4. Características de los usuarios

Rol	Descripción	Nivel técnico esperado
Administrador (ROLE_ADMIN)	Supervisa roles, operatividad general e inventario/facturación cuando estos módulos existan.	Usuario de negocio, sin conocimientos técnicos.

Veterinario (ROLE_VETERINARIO)	Registra diagnósticos y gestiona historiales clínicos; en el alcance actual, gestiona mascotas.	Usuario de negocio, sin conocimientos técnicos.
Auxiliar (ROLE_AUXILIAR)	Apoya operaciones administrativas y de recepción; puede crear y editar mascotas.	Usuario de negocio, sin conocimientos técnicos.
Dueño de mascota (ROLE_DUENO)	Accede en modo de lectura al seguimiento de su(s) mascota(s).	Usuario externo, uso ocasional desde navegador o móvil.

2.5. Restricciones

- El proyecto se desarrolla dentro del calendario académico del PFC (semanas 5 a 17 del período 2026-2027 PPA); cada entrega tiene alcance y fecha de cierre fijos.
- El backend debe ejecutarse sobre Java 21 LTS y Spring Boot 3.2.x; el frontend sobre Angular 17 o superior.
- La persistencia de datos usa exclusivamente PostgreSQL 16, con esquema versionado por Flyway; queda prohibido `spring.jpa.hibernate.ddl-auto=update`.
- Toda contraseña se almacena con hash BCrypt; ninguna credencial se guarda en texto plano.
- El sistema completo debe poder desplegarse con un único comando (`make up`) desde una clonación limpia del repositorio, con imágenes Docker fijadas por `digest sha256`.

2.6. Supuestos y dependencias

- Se asume disponibilidad de un entorno con Docker y Docker Compose para el despliegue reproducible.
- Se asume que los servicios externos de IoT e IA, cuando se incorporen, expondrán APIs REST o colas de mensajería compatibles con el backend Spring Boot.
- La continuidad de los requisitos Should/Could depende de la disponibilidad de tiempo del equipo en las semanas restantes del calendario académico, según lo señalado en el plan de trabajo de la Entrega 1A.

3. Requisitos específicos

Cada requisito individual cumple las nueve características de calidad C1–C9 del INCOSE Guide to Writing Requirements v4 (Necessary, Appropriate, Unambiguous, Complete, Singular, Feasible, Verifiable, Correct, Conforming). El enunciado sigue el patrón [condición] [sujeto] shall [acción] [objeto] [restricción] recomendado por ISO/IEC/IEEE 29148.

3.1. Requisitos funcionales — Módulo de Autenticación y Autorización (implementado)

Campo	Contenido
Identificador	REQ-F-001
Nombre	Registro de usuario
Descripción	Al recibir una solicitud de registro con nombre, correo electrónico y contraseña válidos, el sistema deberá crear un nuevo usuario con rol ROLE_DUENO por defecto, ignorando cualquier rol enviado por el cliente en la solicitud.
Rationale	Evita que un usuario público se autoasigne privilegios administrativos (escalada de privilegios), riesgo detectado y corregido durante la Entrega 1B.
Prioridad (MoSCoW)	Must
Criterio de aceptación	Una solicitud POST /api/auth/registro con datos válidos responde 201 Created y el usuario queda persistido con rol ROLE_DUENO independientemente del campo rol enviado.
Método de verificación	Test — AuthControllerTest (JUnit 5 + MockMvc).
Estado	Implementado y verificado.

Campo	Contenido
Identificador	REQ-F-002
Nombre	Rechazo de correo duplicado
Descripción	Al recibir una solicitud de registro cuyo correo electrónico ya exista en el sistema, el sistema deberá rechazar la operación y responder con un error 409 con formato ProblemDetails.
Rationale	Garantiza la unicidad del correo como identificador de inicio de sesión y evita cuentas duplicadas.
Prioridad (MoSCoW)	Must
Criterio de aceptación	Una solicitud de registro con correo ya existente responde 409 con cuerpo ProblemDetails (RFC 7807).
Método de verificación	Test — EmailDuplicadoException + GlobalExceptionHandler.
Estado	Implementado y verificado.

Campo	Contenido
Identificador	REQ-F-003
Nombre	Inicio de sesión con emisión de tokens
Descripción	Al recibir credenciales válidas de correo y contraseña, el sistema deberá autenticar al usuario y emitir un access token y un refresh token firmados, ambos en formato JWT conforme a RFC 7519.
Rationale	Sostiene el flujo de autenticación stateless que fundamenta toda la autorización del sistema.
Prioridad (MoSCoW)	Must
Criterio de aceptación	POST /api/auth/login con credenciales válidas responde 200 con un access token de vigencia de una hora y un refresh token, ambos con los siete claims estándar (iss, sub, aud, exp, nbf, iat, jti).
Método de verificación	Test — AuthControllerTest; inspección de claims con JwtServiceTest.
Estado	Implementado y verificado.

Campo	Contenido
Identificador	REQ-F-004
Nombre	Renovación de token
Descripción	Al recibir un refresh token válido y no revocado, el sistema deberá emitir un nuevo access token sin requerir el ingreso de credenciales.
Rationale	Permite sesiones prolongadas sin exponer las credenciales del usuario en cada solicitud.

Prioridad (MoSCoW)	Must
Criterio de aceptación	POST /api/auth/refresh con un refresh token válido responde 200 con un nuevo access token.
Método de verificación	Test — AuthControllerTest.
Estado	Implementado y verificado.

Campo	Contenido
Identificador	REQ-F-005
Nombre	Cierre de sesión con revocación
Descripción	Al recibir una solicitud de cierre de sesión con un token vigente, el sistema deberá registrar el identificador jti del token en una lista negra en Redis con un tiempo de vida igual al tiempo restante hasta su expiración.
Rationale	La naturaleza stateless de JWT exige un mecanismo explícito de revocación para que el cierre de sesión sea efectivo antes de la expiración natural del token (ADR-003).
Prioridad (MoSCoW)	Must
Criterio de aceptación	Tras un POST /api/auth/logout exitoso, una solicitud posterior con el mismo token a un endpoint protegido responde 401.
Método de verificación	Test — AuthControllerTest; inspección de TokenBlacklistService.
Estado	Implementado y verificado.

Campo	Contenido
Identificador	REQ-F-006
Nombre	Control de acceso por rol
Descripción	Al recibir una solicitud a un recurso protegido, el sistema deberá verificar el rol del usuario autenticado contra los roles autorizados para ese recurso y rechazar la solicitud con 403 si el rol no está autorizado.
Rationale	Materializa el requisito heredado RF-13 de la Entrega 1A (control de acceso por rol) sobre la implementación real con Spring Security y anotaciones @PreAuthorize.
Prioridad (MoSCoW)	Must
Criterio de aceptación	Un usuario con rol ROLE_DUEÑO que intenta ejecutar POST /api/mascotas recibe 403 Forbidden.
Método de verificación	Test de integración; auditoría OWASP A01 (Bloque C.2 de la Guía).
Estado	Implementado y verificado.

Campo	Contenido
Identificador	REQ-F-007
Nombre	Consulta del perfil propio
Descripción	Al recibir una solicitud autenticada a GET /api/usuarios/me, el sistema deberá devolver los datos del perfil del usuario correspondiente al token presentado.
Rationale	Necesario para que el frontend muestre el nombre y rol del usuario en sesión sin exponer el listado completo de usuarios.
Prioridad (MoSCoW)	Should
Criterio de aceptación	GET /api/usuarios/me con un token válido responde 200 con nombre, correo y rol del usuario autenticado.
Método de verificación	Inspección de UsuarioController; prueba manual con Postman.
Estado	Implementado.

3.2. Requisitos funcionales — Módulo de Mascotas (implementado)

Campo	Contenido
Identificador	REQ-F-008
Nombre	Registro de mascota
Descripción	Al recibir una solicitud con nombre, especie, raza y fecha de nacimiento válidos, el sistema deberá crear una mascota asociada al dueño indicado y devolver el registro creado.
Rationale	Actualiza y reemplaza el requisito heredado RF-01 de la Entrega 1A, adaptándolo a la implementación real vía API REST en lugar de formulario ASP.NET.
Prioridad (MoSCoW)	Must
Criterio de aceptación	POST /api/mascotas con datos válidos y rol autorizado (ADMIN, VETERINARIO o AUXILIAR) responde 201 Created y la mascota queda vinculada al dueño_id indicado.

Método de verificación	Test — MascotaControllerTest.
Estado	Implementado y verificado.

Campo	Contenido
Identificador	REQ-F-009
Nombre	Listado paginado de mascotas
Descripción	Al recibir una solicitud de listado, el sistema deberá devolver las mascotas activas de forma paginada, accesible para los cuatro roles del sistema.
Rationale	Actualiza el requisito heredado RF-02 de la Entrega 1A (consulta de dueños de mascotas), reformulado para reflejar que la consulta implementada es sobre la entidad Mascota.
Prioridad (MoSCoW)	Must
Criterio de aceptación	GET /api/mascotas responde 200 con una página de resultados y metadatos de paginación (Pageable de Spring Data).
Método de verificación	Test — MascotaControllerTest.
Estado	Implementado y verificado.

Campo	Contenido
Identificador	REQ-F-010
Nombre	Consulta de mascota por identificador
Descripción	Al recibir una solicitud con un identificador de mascota existente, el sistema deberá devolver los datos completos de esa mascota.
Rationale	Complementa REQ-F-009 para el caso de consulta puntual, necesario antes de una edición o de mostrar el detalle en la interfaz.
Prioridad (MoSCoW)	Must
Criterio de aceptación	GET /api/mascotas/{id} con un identificador existente responde 200 con los datos de la mascota; con un identificador inexistente responde 404 con ProblemDetails.
Método de verificación	Test — MascotaControllerTest; RecursoNoEncontradoException.
Estado	Implementado y verificado.

Campo	Contenido
Identificador	REQ-F-011
Nombre	Actualización de mascota
Descripción	Al recibir una solicitud de actualización con datos válidos sobre una mascota existente, el sistema deberá modificar los atributos indicados y registrar la fecha de actualización.
Rationale	Actualiza el requisito heredado RF-02 de la Entrega 1A en su componente de edición.
Prioridad (MoSCoW)	Must
Criterio de aceptación	PUT /api/mascotas/{id} con datos válidos y rol autorizado responde 200 con los datos actualizados; el campo actualizado_en refleja la hora de la operación.
Método de verificación	Test — MascotaControllerTest.
Estado	Implementado y verificado.

Campo	Contenido
Identificador	REQ-F-012
Nombre	Baja lógica de mascota
Descripción	Al recibir una solicitud de eliminación sobre una mascota existente, el sistema deberá marcar el registro como inactivo sin eliminarlo físicamente de la base de datos.
Rationale	Preserva el historial e integridad referencial de futuros módulos (citas, historial clínico) que dependerán de la mascota.
Prioridad (MoSCoW)	Must
Criterio de aceptación	DELETE /api/mascotas/{id} con rol autorizado responde 204 No Content y el registro pasa a activo = false; ya no aparece en el listado por defecto.
Método de verificación	Test — MascotaControllerTest.
Estado	Implementado y verificado.

3.3. Requisitos funcionales — Módulos pendientes (heredados de la Entrega 1A, no implementados)

Los siguientes requisitos provienen del corpus original de la Entrega 1A. Se mantienen porque siguen representando la visión del producto y porque el modelo de datos que los sustenta (Cita, Historial_Clinico, Dispositivo_IoT, Chat_Triage, Factura, Producto_Servicio, Proveedor) ya fue diseñado y documentado en el diagrama entidad-relación de dicha entrega. Se reclasifican con prioridad MoSCoW ajustada a su condición de pendientes y se declaran en estado "pendiente" en la matriz de trazabilidad, conforme exige el bloque A.3.3 de la Guía de la Tercera Entrega.

Campo	Contenido
Identificador	REQ-F-013
Nombre	Registro de atención médica e historial clínico
Descripción	Cuando un veterinario registre una atención médica, el sistema deberá almacenar diagnóstico, tratamiento y observaciones, asociándolos cronológicamente al historial clínico de la mascota.
Rationale	Actualiza los requisitos heredados RF-03 y RF-04 de la Entrega 1A; sustenta la razón de ser original del proyecto (fragmentación de la información clínica).
Prioridad (MoSCoW)	Should
Criterio de aceptación	Al consultar el historial de una mascota, las atenciones se muestran ordenadas por fecha, cada una con su diagnóstico y tratamiento.
Método de verificación	Pendiente de implementación; se verificará por test de integración cuando el módulo exista.
Estado	Pendiente — diseño de datos disponible (tabla Historial_Clinico), sin código.

Campo	Contenido
Identificador	REQ-F-014
Nombre	Prescripción de medicamentos
Descripción	Cuando se registre una atención médica, el sistema deberá permitir asociar uno o más medicamentos prescritos a esa atención.
Rationale	Actualiza el requisito heredado RF-05 de la Entrega 1A.
Prioridad (MoSCoW)	Could
Criterio de aceptación	Los medicamentos ingresados en una atención quedan vinculados a dicha atención y son visibles en el historial.
Método de verificación	Pendiente de implementación.
Estado	Pendiente.

Campo	Contenido
Identificador	REQ-F-015
Nombre	Gestión de citas veterinarias
Descripción	El sistema deberá permitir registrar, modificar y consultar citas veterinarias mediante un calendario interactivo, asociando cada cita a una mascota y a un veterinario.
Rationale	Actualiza el requisito heredado RF-06 de la Entrega 1A; el modelo de datos (tabla Cita) ya está diseñado.
Prioridad (MoSCoW)	Should
Criterio de aceptación	Las citas creadas aparecen en el calendario y pueden modificarse o cancelarse posteriormente.
Método de verificación	Pendiente de implementación.
Estado	Pendiente — diseño de datos disponible, sin código.

Campo	Contenido
Identificador	REQ-F-016
Nombre	Recepción de telemetría de dispositivos IoT
Descripción	El backend deberá exponer una API para recibir datos de ubicación enviados por dispositivos de rastreo asociados a las mascotas.
Rationale	Actualiza el requisito heredado RF-08/RF-09 de la Entrega 1A; corresponde a la visión de negocio de seguimiento preventivo.
Prioridad (MoSCoW)	Could

Criterio de aceptación	La API recibe coordenadas válidas y las almacena junto con la fecha y hora de actualización; la ubicación es visualizable en un mapa.
Método de verificación	Pendiente de implementación.
Estado	Pendiente — diseño de datos disponible (tabla Dispositivo IoT), sin código.

Campo	Contenido
Identificador	REQ-F-017
Nombre	Recomendaciones clínicas asistidas por IA
Descripción	El sistema deberá generar recomendaciones clínicas informativas basadas en los datos del historial médico de la mascota, mediante un servicio de Inteligencia Artificial.
Rationale	Actualiza el requisito heredado RF-10 de la Entrega 1A. Depende funcionalmente de que exista historial clínico (REQ-F-013).
Prioridad (MoSCoW)	Could
Criterio de aceptación	El veterinario puede visualizar una recomendación generada automáticamente para una mascota con historial registrado.
Método de verificación	Pendiente de implementación.
Estado	Pendiente — depende de REQ-F-013.

Campo	Contenido
Identificador	REQ-F-018
Nombre	Facturación digital y registro de pagos
Descripción	El sistema deberá generar comprobantes de pago digitales en formato PDF y permitir registrar el método de pago asociado a un servicio veterinario.
Rationale	Actualiza los requisitos heredados RF-11 y RF-12 de la Entrega 1A; el modelo de datos (Factura, Detalle_Factura, Producto_Servicio) ya está diseñado, incluyendo campos de integración con el SRI ecuatoriano.
Prioridad (MoSCoW)	Should
Criterio de aceptación	El usuario puede descargar un PDF válido después de registrar un servicio veterinario; la forma de pago queda almacenada junto con el servicio.
Método de verificación	Pendiente de implementación.
Estado	Pendiente — diseño de datos disponible, sin código.

Campo	Contenido
Identificador	REQ-F-019
Nombre	Generación de reportes exportables
Descripción	El sistema deberá generar reportes estadísticos exportables en formato PDF y Excel.
Rationale	Actualiza el requisito heredado RF-14 de la Entrega 1A.
Prioridad (MoSCoW)	Could
Criterio de aceptación	El usuario puede descargar correctamente los reportes generados en ambos formatos.
Método de verificación	Pendiente de implementación.
Estado	Pendiente.

Campo	Contenido
Identificador	REQ-F-020
Nombre	Auditoría de operaciones
Descripción	El sistema deberá registrar las acciones relevantes realizadas por los usuarios, incluyendo usuario, fecha y hora de cada operación.
Rationale	Actualiza el requisito heredado RF-15 de la Entrega 1A; se relaciona con el requisito no funcional REQ-NF-009 (bloque C.2, control A09 de OWASP) ya cubierto parcialmente por el logging de autenticación.
Prioridad (MoSCoW)	Should
Criterio de aceptación	Cada operación relevante (creación, modificación, eliminación) queda registrada con identificador de usuario, fecha y hora.
Método de verificación	Pendiente de implementación de auditoría genérica; el logging de autenticación (login exitoso/fallido) ya está parcialmente cubierto.
Estado	Parcialmente pendiente.

3.4. Requisitos no funcionales

Los requisitos no funcionales heredados de la Entrega 1A (RNF-01 a RNF-08 y RNF-WEB-01 a RNF-WEB-05) se consolidan y renombran con el prefijo REQ-NF, siguiendo el mismo patrón de campos que los requisitos funcionales, e incorporando los umbrales cuantitativos exigidos por el Bloque C de la Guía de la Tercera Entrega.

Campo	Contenido
Identificador	REQ-NF-001
Categoría	Rendimiento
Nombre	Tiempo de respuesta con caché caliente
Descripción	El endpoint de listado de mascotas deberá responder con un percentil 95 de tiempo de respuesta inferior a 200 ms cuando el resultado se sirva desde caché.
Rationale	Actualiza RNF-01/RNF-WEB-04 de la Entrega 1A con el umbral cuantitativo exigido por el bloque C.1 de la Guía y alineado a ISO/IEC 25010 (Eficiencia de desempeño).
Prioridad (MoSCoW)	Must
Criterio de aceptación	Corrida de carga con k6 (50 VUs, 30 s) muestra p95 < 200 ms con caché caliente en al menos 3 corridas independientes.
Método de verificación	Test de rendimiento (k6) — pendiente de ejecutar y archivar en docs/mediciones/perf/.
Estado	Pendiente de evidencia empírica.

Campo	Contenido
Identificador	REQ-NF-002
Categoría	Rendimiento
Nombre	Tiempo de respuesta con caché fría
Descripción	El endpoint de listado de mascotas deberá responder con un percentil 95 de tiempo de respuesta inferior a 500 ms cuando el resultado no esté disponible en caché.
Rationale	Complementa REQ-NF-001 para el caso sin caché, según el umbral declarado en el bloque C.1 de la Guía.
Prioridad (MoSCoW)	Must
Criterio de aceptación	Corrida de carga con k6 muestra p95 < 500 ms con caché fría.
Método de verificación	Test de rendimiento (k6) — pendiente de ejecutar.
Estado	Pendiente de evidencia empírica.

Campo	Contenido
Identificador	REQ-NF-003
Categoría	Seguridad
Nombre	Cifrado en tránsito
Descripción	Toda comunicación entre cliente y servidor deberá realizarse mediante HTTPS con TLS 1.3.
Rationale	Actualiza RNF-02 de la Entrega 1A. Protege credenciales y tokens JWT frente a interceptación en tránsito.
Prioridad (MoSCoW)	Must
Criterio de aceptación	curl -v contra el endpoint muestra negociación TLSv1.3 con suite AEAD; toda solicitud HTTP se redirige a HTTPS.
Método de verificación	Auditoría OWASP A02 (Bloque C.2 de la Guía) — pendiente de captura.
Estado	Pendiente de evidencia empírica en el entorno de despliegue objetivo.

Campo	Contenido
Identificador	REQ-NF-004
Categoría	Seguridad
Nombre	Gestión de tokens JWT
Descripción	La autenticación deberá gestionarse mediante tokens JWT firmados con HS256, con expiración configurable y los siete claims estándar (iss, sub, aud, exp, nbf, iat, jti).
Rationale	Actualiza RNF-03/RNF-WEB-03 de la Entrega 1A. La inclusión del claim aud responde a la observación de la Entrega 1B sobre restringir el público objetivo del token.
Prioridad (MoSCoW)	Must
Criterio de aceptación	Todo token emitido incluye los siete claims; un token con audiencia distinta a la esperada es rechazado por el filtro de autenticación.
Método de verificación	Test — JwtServiceTest.

Estado	Implementado y verificado.
--------	----------------------------

Campo	Contenido
Identificador	REQ-NF-005
Categoría	Seguridad
Nombre	Almacenamiento seguro de contraseñas
Descripción	Las contraseñas de los usuarios deberán almacenarse cifradas mediante el algoritmo BCrypt con factor de costo 12.
Rationale	Actualiza RNF-07 de la Entrega 1A con el parámetro concreto implementado.
Prioridad (MoSCoW)	Must
Criterio de aceptación	Inspección de la tabla usuarios confirma que ningún valor de password_hash corresponde a texto plano; el prefijo del hash corresponde a BCrypt.
Método de verificación	Inspección de base de datos; revisión de código (UserDetailsServiceImpl).
Estado	Implementado y verificado.

Campo	Contenido
Identificador	REQ-NF-006
Categoría	Seguridad
Nombre	Formato uniforme de errores
Descripción	Toda respuesta de error de la API deberá seguir el formato ProblemDetails de RFC 7807, incluyendo los campos type, title, status y detail.
Rationale	Estandariza el manejo de errores del backend y facilita su consumo por el frontend; exigido por el bloque A.1 de la Guía.
Prioridad (MoSCoW)	Must
Criterio de aceptación	El 100 % de las respuestas de error del backend contienen los campos type, title, status y detail en formato JSON conforme a RFC 7807.
Método de verificación	Revisión de código — GlobalExceptionHandler y ProblemDetailFactory; prueba con colección Postman.
Estado	Implementado y verificado.

Campo	Contenido
Identificador	REQ-NF-007
Categoría	Usabilidad
Nombre	Diseño responsivo
Descripción	La interfaz deberá adaptarse correctamente a dispositivos móviles, tablets y equipos de escritorio, en el rango de 320px a 1440px de ancho de viewport.
Rationale	Actualiza RNF-04/RNF-WEB-01 de la Entrega 1A.
Prioridad (MoSCoW)	Should
Criterio de aceptación	Verificación visual en 320px, 768px y 1440px sin desbordamiento de contenido ni pérdida de funcionalidad.
Método de verificación	Auditoría Lighthouse (Bloque C.5) — pendiente de ejecutar sobre el frontend Angular.
Estado	Pendiente de evidencia empírica; frontend actual limitado a login y listado de mascotas.

Campo	Contenido
Identificador	REQ-NF-008
Categoría	Compatibilidad
Nombre	Soporte de navegadores
Descripción	La aplicación deberá funcionar correctamente en las versiones modernas de Chrome, Firefox y Edge.
Rationale	Actualiza RNF-05 de la Entrega 1A.
Prioridad (MoSCoW)	Should
Criterio de aceptación	Ejecución satisfactoria de las pruebas funcionales manuales en los tres navegadores.
Método de verificación	Prueba manual exploratoria — pendiente de documentar.
Estado	Pendiente de evidencia.

Campo	Contenido
Identificador	REQ-NF-009
Categoría	Seguridad

Nombre	Registro de eventos de autenticación
Descripción	El sistema deberá registrar cada intento de inicio de sesión, exitoso o fallido, incluyendo dirección IP, marca de tiempo y el sub del JWT cuando aplique.
Rationale	Corresponde al control OWASP A09 (fallo de registro y monitoreo) exigido por el bloque C.2 de la Guía; también sustenta el requisito funcional heredado de auditoría (REQ-F-020).
Prioridad (MoSCoW)	Must
Criterio de aceptación	El archivo de log contiene entradas de login exitoso y fallido con IP, timestamp y sub.
Método de verificación	Auditoría OWASP A09 — pendiente de capturar evidencia archivada en docs/mediciones/sec/.
Estado	Pendiente de evidencia empírica; mecanismo de logging por confirmar en el código actual.

Campo	Contenido
Identificador	REQ-NF-010
Categoría	Seguridad
Nombre	Limitación de intentos de inicio de sesión
Descripción	El sistema deberá responder 429 Too Many Requests a partir del sexto intento fallido consecutivo de inicio de sesión desde la misma dirección IP.
Rationale	Corresponde al control OWASP A07 (fallo de identificación y autenticación) exigido por el bloque C.2 de la Guía.
Prioridad (MoSCoW)	Should
Criterio de aceptación	Seis intentos fallidos consecutivos desde la misma IP producen 429 a partir del sexto intento.
Método de verificación	Auditoría OWASP A07 — pendiente de implementar y evidenciar.
Estado	Pendiente de implementación (rate limiting no presente en el código actual).

Campo	Contenido
Identificador	REQ-NF-011
Categoría	Disponibilidad
Nombre	Disponibilidad durante evaluación académica
Descripción	El sistema deberá estar disponible y desplegable mediante un único comando durante las semanas de evaluación establecidas por la asignatura.
Rationale	Actualiza RNF-06 de la Entrega 1A; se refuerza con el requisito de reproducibilidad automática del bloque B.1 de la Guía.
Prioridad (MoSCoW)	Must
Criterio de aceptación	make up levanta los cuatro servicios (postgres, redis, backend, frontend) como healthy desde una clonación limpia en menos de dos minutos.
Método de verificación	Prueba de reproducibilidad — pendiente de validar por un tercero externo al equipo.
Estado	Implementado (Makefile y docker-compose.yml existentes); validación por tercero pendiente.

Campo	Contenido
Identificador	REQ-NF-012
Categoría	Mantenibilidad
Nombre	Arquitectura en capas
Descripción	La arquitectura del backend deberá estar organizada en capas separadas de controladores, servicios, repositorios y entidades.
Rationale	Actualiza RNF-08 de la Entrega 1A, reflejando la organización de paquetes efectivamente implementada (controller, service, repository, entity, dto, security, exception).
Prioridad (MoSCoW)	Must
Criterio de aceptación	Revisión de la estructura de paquetes del backend confirma la separación descrita, sin lógica de negocio en los controladores.
Método de verificación	Revisión de código.
Estado	Implementado y verificado.

Campo	Contenido
Identificador	REQ-NF-013
Categoría	Mantenibilidad
Nombre	Estrategia de acceso a datos híbrida

Descripción	El sistema deberá implementar los CRUD elementales sobre entidades JPA usando Spring Data, y encapsular en procedimientos almacenados o funciones SQL versionados toda operación que involucre joins, agregaciones, reportes o validaciones cruzadas entre tablas.
Rationale	Exigido por el bloque A.2 de la Guía de la Tercera Entrega para garantizar defensas equivalentes contra inyección SQL en ambos mecanismos de acceso a datos.
Prioridad (MoSCoW)	Must
Criterio de aceptación	Ningún fragmento de JPQL, HQL o SQL nativo concatena entrada de usuario; al menos un procedimiento almacenado existe, está invocado desde un repositorio Spring Data y catalogado en docs/basedatos/CATALOGO-SP.md.
Método de verificación	Auditoría estática (scripts/audit-sql-dynamic.sh) — pendiente de crear; actualmente no existen operaciones que requieran procedimientos almacenados porque solo hay CRUD elemental sobre dos entidades.
Estado	Cumplido por alcance actual (no aplica todavía); catálogo de SP pendiente de crearse para cuando existan reportes o joins.

4. Arquitectura de la aplicación web

La arquitectura general (modelo multinivel, SPA + API REST + base de datos relacional) se mantiene respecto a la Entrega 1A. Los diagramas de esta sección se actualizaron durante la Entrega 1B para reflejar la pila tecnológica real (Java/Spring Boot en lugar de ASP.NET Core) y se conservan vigentes para esta versión.

4.1. Diagrama de contexto (C4 — Nivel 1)

Se conserva el diagrama de contexto de la Entrega 1A: BIOPET interactúa con Veterinarios, Administradores y Dueños de mascotas, y se proyecta su comunicación con sistemas externos de IoT, IA y correo electrónico. Este nivel no depende del lenguaje de implementación del backend y por tanto no requirió actualización.

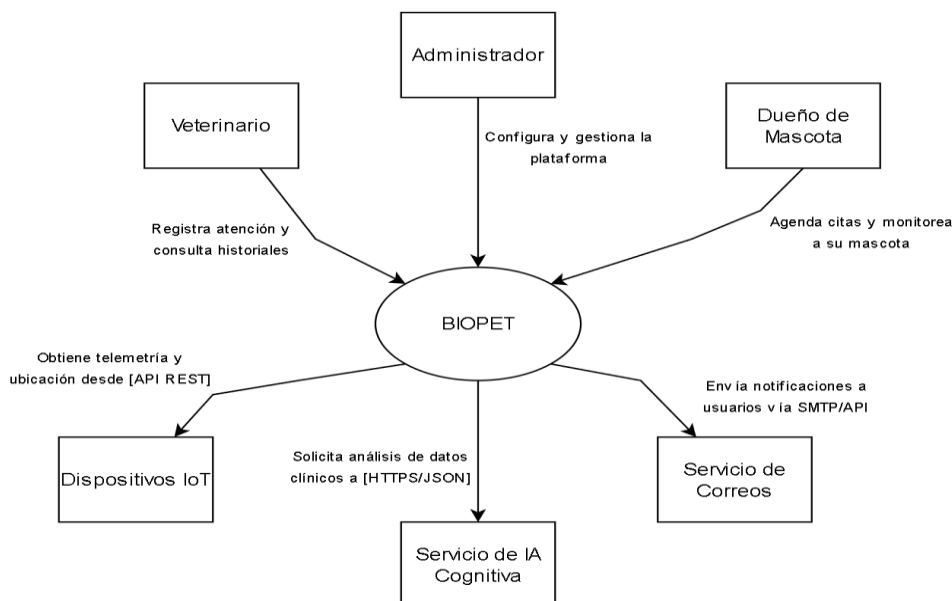


Figura 1. Diagrama de contexto (C4 Nivel 1) — heredado de la Entrega 1A.

4.2. Diagrama de contenedores (C4 — Nivel 2)

Actualizado en la Entrega 1B para reflejar el backend en Spring Boot, la base de datos PostgreSQL con Flyway y la caché Redis para la lista negra de tokens.

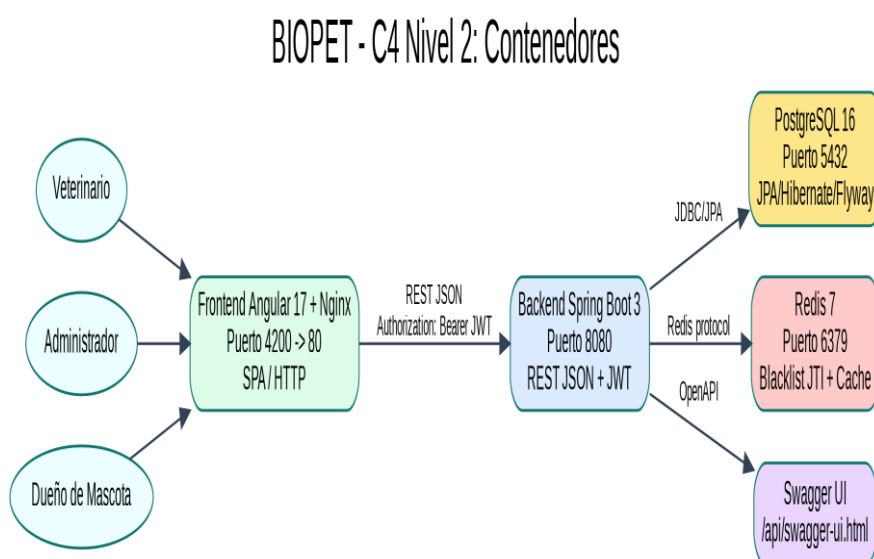


Figura 2. Diagrama de contenedores (C4 Nivel 2) — vigente, Entrega 1B.

4.3. Diagrama de clases

Representa las clases del dominio implementadas (Usuario, Mascota, Rol) junto con los componentes de seguridad (JwtService, JwtAuthenticationFilter, TokenBlacklistService).

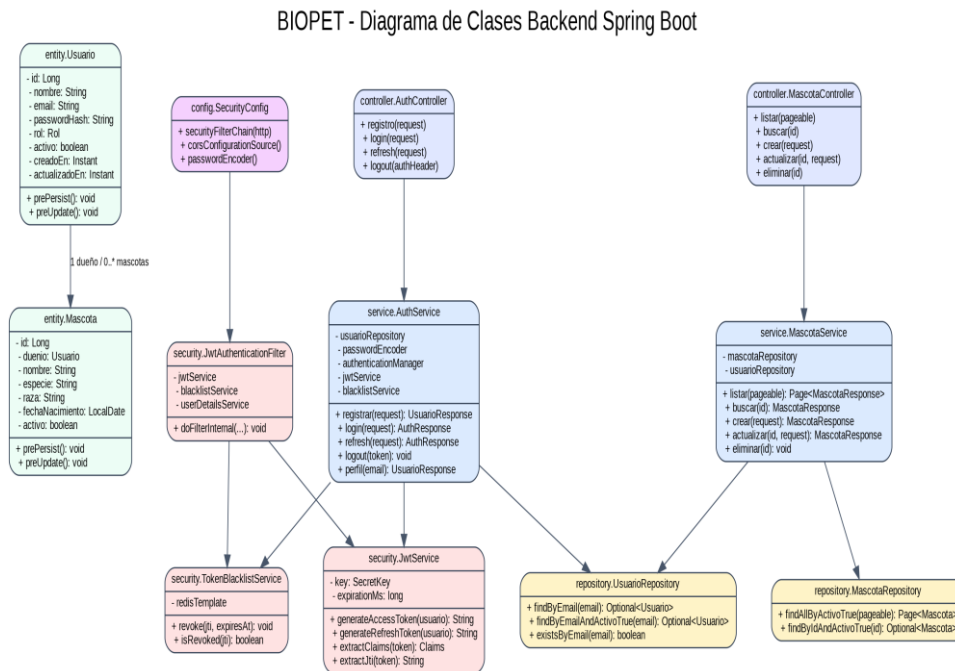


Figura 3. Diagrama de clases — Entrega 1B.

4.4. Diagrama de secuencia: flujo de autenticación JWT

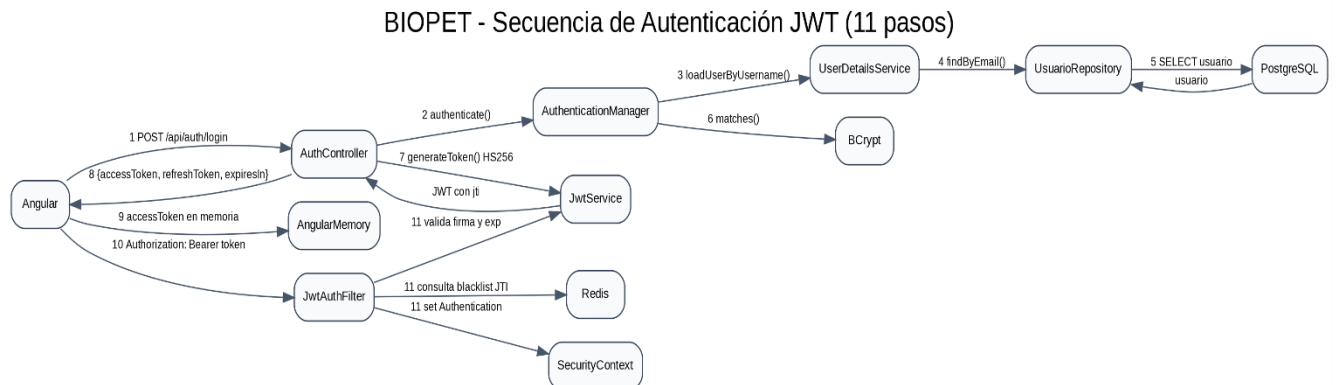


Figura 4. Diagrama de secuencia del flujo de autenticación — Entrega 1B.

4.5. Descripción de capas y tecnologías (actualizada)

Capa	Tecnología (v0.9.0-rc)	Tecnología en SRS v0.3.0 (superada)
Backend	Java 21 LTS + Spring Boot 3.2.x + Spring Security 6	ASP.NET Core 8 (C#)
Acceso a datos	Spring Data JPA + Hibernate, migraciones Flyway	Entity Framework Core
Base de datos	PostgreSQL 16	PostgreSQL (sin cambio)
Caché	Redis 7 (lista negra de JTI)	No contemplado en v0.3.0
Autenticación	JWT (jjwt 0.12) firmado HS256, stateless	JWT (sin biblioteca específica definida)
Frontend	Angular 17+ (esqueleto: login, guard, interceptor, listado de mascotas)	HTML5 + CSS3 + JS ES6 + Bootstrap 5
Documentación de API	Springdoc OpenAPI 2.x en /api/docs	No especificado
Contenedores	Docker Compose (postgres, redis, backend, frontend) con healthchecks	Docker (mención general) / XAMPP local
Control de versiones	Git + GitHub, Conventional Commits	Git + GitHub (sin convención declarada)

El cambio de ASP.NET Core a Spring Boot no está documentado en el repositorio provisto mediante un ADR dedicado; se recomienda generar un ADR de reemplazo de ADR-001 (ver observación en la sección 7) para dejar constancia formal del contexto, las alternativas y las consecuencias de la migración, tal como exige el bloque D de la Guía de la Tercera Entrega.

5. Modelo de base de datos

El diagrama entidad-relación completo, diseñado en la Entrega 1A, se mantiene vigente como visión de datos del producto. A la fecha de esta versión, únicamente las entidades Usuario y Mascota están materializadas en el esquema de base de datos mediante la migración Flyway V1__schema_inicial.sql; el resto de entidades (Cita, Historial_Clinico, Dispositivo_IoT, Chat_Triage, Producto_Servicio, Factura, Detalle_Factura, Proveedor, Mercaderia, Detalle_Ingreso) permanecen en estado de diseño, sin migración correspondiente.

BIOPET - Diagrama Entidad-Relación Entrega 1B

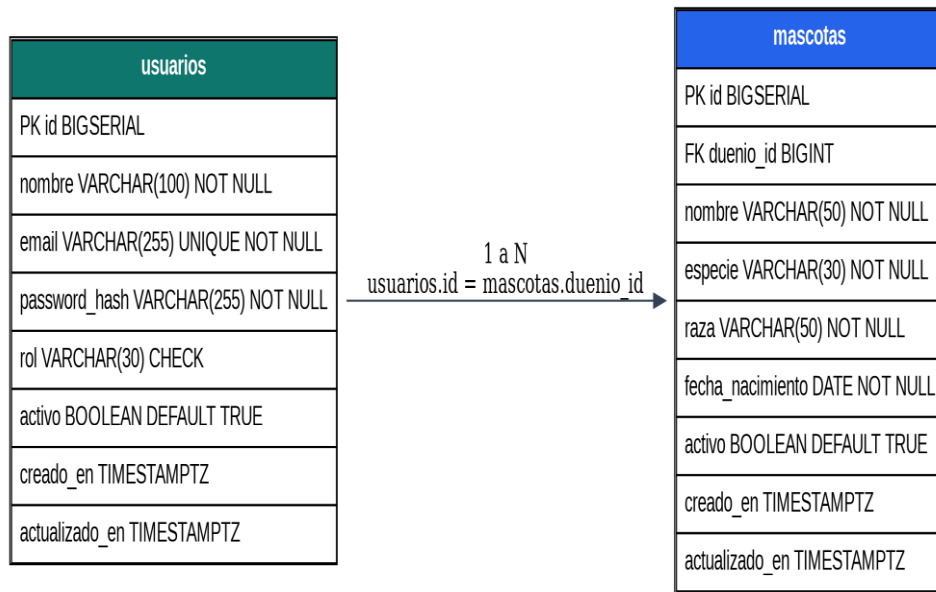


Figura 5. Diagrama entidad-relación (DER) completo del dominio BIOPET — visión de producto, Entrega 1B.

5.1. Diccionario de datos — Entidades implementadas

5.1.1. usuarios

Atributo	Tipo de dato	Nulo	Restricción	Descripción
id	BIGSERIAL	No	PK	Identificador único del usuario.
nombre	VARCHAR(100)	No		Nombre completo del usuario.
email	VARCHAR(255)	No	UNIQUE	Correo electrónico, usado como identificador de inicio de sesión.
password_hash	VARCHAR(255)	No		Hash BCrypt de la contraseña. Nunca se serializa en las respuestas de la API.
rol	VARCHAR(30)	No	CHECK IN (ROLE_ADMIN, ROLE_VETERINARIO, ROLE_DUENO, ROLE_AUXILIAR)	Rol del usuario para control de acceso.
activo	BOOLEAN	No	DEFAULT TRUE	Bandera de baja lógica.
creado_en	TIMESTAMPTZ	No	DEFAULT NOW()	Fecha y hora de creación del registro.
actualizado_en	TIMESTAMPTZ	No	DEFAULT NOW(), trigger	Fecha y hora de la última modificación, actualizada por trigger.

5.1.2. mascotas

Atributo	Tipo de dato	Nulo	Restricción	Descripción
id	BIGSERIAL	No	PK	Identificador único de la mascota.
duenio_id	BIGINT	No	FK → usuarios(id)	Referencia al usuario dueño de la mascota.
nombre	VARCHAR(50)	No		Nombre de la mascota.
especie	VARCHAR(30)	No		Especie (p. ej. Canino, Felino).
raza	VARCHAR(50)	No		Raza específica de la mascota.
fecha_nacimiento	DATE	No		Fecha de nacimiento registrada.
activo	BOOLEAN	No	DEFAULT TRUE	Bandera de baja lógica.
creado_en / actualizado_en	TIMESTAMPTZ	No	DEFAULT NOW(), trigger	Auditoría temporal del registro.

5.2. Entidades pendientes de materialización

Documentadas íntegramente en el diccionario de datos de la Entrega 1A (Cita, Historial_Clinico, Dispositivo_IoT, Chat_Triage, Producto_Servicio, Factura, Detalle_Factura, Proveedor, Mercaderia, Detalle_Ingreso) y en el script DDL de referencia allí incluido. No se reproducen íntegramente en este documento por no haber cambiado; se referencia el documento original y se remite a docs/diagramas/MER.png del repositorio de la Entrega 1A para el diseño completo. Cuando se implementen, cada migración Flyway correspondiente deberá versionarse como V2__, V3__, etc., sin modificar V1__schema_inicial.sql.

6. Matriz de trazabilidad (resumen)

La matriz completa se mantiene en docs/trazabilidad/matriz.csv, con las columnas exigidas por el bloque A.3.3 de la Guía: id_requisito, tipo, prioridad_moscow, historia_usuario, caso_de_uso, modulo_codigo, endpoint_api, prueba_automatizada, tipo_acceso, evidencia_empirica y estado. La siguiente tabla resume, por requisito Must, el módulo de código y el endpoint asociado.

Requisito	Módulo de código	Endpoint / componente	Prueba	Estado
REQ-F-001	AuthController / AuthService	POST /api/auth/registro	AuthControllerTest	verificado
REQ-F-002	GlobalExceptionHandler	POST /api/auth/registro (409)	AuthControllerTest	verificado
REQ-F-003	AuthController / JwtService	POST /api/auth/login	AuthControllerTest, JwtServiceTest	verificado
REQ-F-004	AuthController / AuthService	POST /api/auth/refresh	AuthControllerTest	verificado
REQ-F-005	AuthController / TokenBlacklistService	POST /api/auth/logout	AuthControllerTest	verificado
REQ-F-006	SecurityConfig / MascotaController	@PreAuthorize en /api/mascotas/**	MascotaControllerTest	verificado
REQ-F-008	MascotaController / MascotaService	POST /api/mascotas	MascotaControllerTest	verificado
REQ-F-009	MascotaController / MascotaService	GET /api/mascotas	MascotaControllerTest	verificado
REQ-F-010	MascotaController / MascotaService	GET /api/mascotas/{id}	MascotaControllerTest	verificado
REQ-F-011	MascotaController / MascotaService	PUT /api/mascotas/{id}	MascotaControllerTest	verificado
REQ-F-012	MascotaController / MascotaService	DELETE /api/mascotas/{id}	MascotaControllerTest	verificado
REQ-NF-004	JwtService	N/A (firma y claims del token)	JwtServiceTest	verificado
REQ-NF-005	UserDetailsServiceImpl	N/A (hash de contraseña)	Inspección	verificado
REQ-NF-006	GlobalExceptionHandler / ProblemDetailFactory	Todos los endpoints	Colección Postman	verificado
REQ-NF-012	Estructura de paquetes backend	N/A	Revisión de código	verificado
REQ-NF-001 / 002	Cache de listado (Redis)	GET /api/mascotas	k6 (pendiente)	pendiente
REQ-NF-003	Nginx / despliegue	N/A	curl -v (pendiente)	pendiente
REQ-NF-009	Logging de autenticación	N/A	Auditoría OWASP A09 (pendiente)	pendiente
REQ-NF-010	Rate limiting (no implementado)	POST /api/auth/login	Auditoría OWASP A07 (pendiente)	pendiente
REQ-NF-011	Makefile / docker-compose.yml	N/A	Validación por tercero (pendiente)	implementado, evidencia pendiente

Los requisitos Should y Could descritos en la sección 3.3 (REQ-F-013 a REQ-F-020) figuran en la matriz completa con estado "pendiente" y sin módulo de código asociado, conforme lo permite explícitamente el bloque A.3.3 de la Guía.

7. Observaciones abiertas e información pendiente

Conforme a la instrucción de no inventar contenido, se listan aquí explícitamente los puntos para los cuales la información disponible en el repositorio y los documentos provistos fue insuficiente para completar el apartado correspondiente con evidencia verificable.

- ADR de cambio de pila tecnológica: el repositorio de la Entrega 1B no contiene un ADR que documente formalmente el reemplazo de ASP.NET Core 8 (ADR-001, Entrega 1A) por Java 21 / Spring Boot 3.2. Solo se encontró ADR-003 (revocación de JWT mediante Redis). Se recomienda crear dicho ADR antes del cierre de la Tercera Entrega, ya que el bloque D de la Guía exige seis ADR con los temas obligatorios, entre ellos la elección de la pila principal.
- Bitácora de observaciones (docs/observaciones/OBSERVACIONES.md, Bloque 0 de la Guía): no se encontró este archivo en el repositorio provisto, por lo que no fue posible verificar qué observaciones puntuales de las Entregas 1A y 1B fueron resueltas ni sus commits de cierre. Este SRS no puede afirmar el porcentaje de observaciones cerradas.
- Evidencia empírica cuantitativa (Bloque C de la Guía): no se encontraron archivos bajo docs/mediciones/ (perf, sec, sus, lighthouse, jacoco) en el repositorio provisto. Los requisitos no funcionales de rendimiento (REQ-NF-001, REQ-NF-002), varios controles OWASP (REQ-NF-003, REQ-NF-009, REQ-NF-010) y los de usabilidad/compatibilidad (REQ-NF-007, REQ-NF-008) se documentan con su umbral objetivo pero se marcan "pendiente de evidencia empírica" porque no existe el archivo de mediciones correspondiente para verificarlos.
- Catálogo de procedimientos almacenados (docs/basedatos/CATALOGO-SP.md, bloque A.2.2 de la Guía): no existen aún operaciones que requieran procedimiento almacenado, dado que el alcance actual se limita a CRUD elemental sobre dos entidades sin joins ni agregaciones. El requisito REQ-NF-013 documenta esta condición; el catálogo deberá crearse en cuanto se implemente el primer reporte o consulta multi-tabla (por ejemplo, al incorporar Citas o Historial Clínico).
- Matriz de trazabilidad completa (docs/trazabilidad/matriz.csv): no se encontró este archivo en el repositorio provisto. La tabla resumen de la sección 6 de este SRS se construyó a partir del código fuente y no reemplaza la matriz formal exigida por el bloque A.3.3 de la Guía, que deberá generarse como archivo CSV independiente.
- Historias de usuario y casos de uso formales (docs/requisitos/historias/ y docs/requisitos/casos-de-uso/, bloque A.3.2 de la Guía): no se encontraron en el repositorio provisto. La matriz de trazabilidad de la sección 6 referencia estos artefactos como pendientes de creación; no fue posible completar las columnas historia_usuario y caso_de_uso con datos verificables.
- Diccionario de datos de mediciones (docs/mediciones/DATA-DICTIONARY.md) y demás entregables del bloque E (LICENSE, CITATION.cff, CONTRIBUTORS.md, CHANGELOG.md, DOI Zenodo): no se encontraron en el repositorio provisto. Quedan fuera del alcance de este SRS, que se limita a la especificación de requisitos, pero se señalan aquí porque su ausencia afecta directamente varios criterios de la rúbrica de la Tercera Entrega (C7, C8, C9).